

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа №2

Выполнила:

Ширкунова Мария

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Улучшить доступность ранее реализованного сайта Мои Финансы. Добавить необходимые HTML-атрибуты ко всему контенту на странице и проверить это с помощью инструментов из Dev Tools браузера Firefox и сервиса Google Lighthouse.

Сервис для управления личными финансами и бюджетом:

- Вход
- Регистрация
- Личный кабинет пользователя (счета, транзакции, бюджеты)
- Поиск и фильтрация транзакций по категории, сумме и дате
- Страница отчёта (графики расходов, категории, прогнозы)
- Интеграция с платёжными аккаунтами (импорт транзакций, настройка правил)

Ход работы

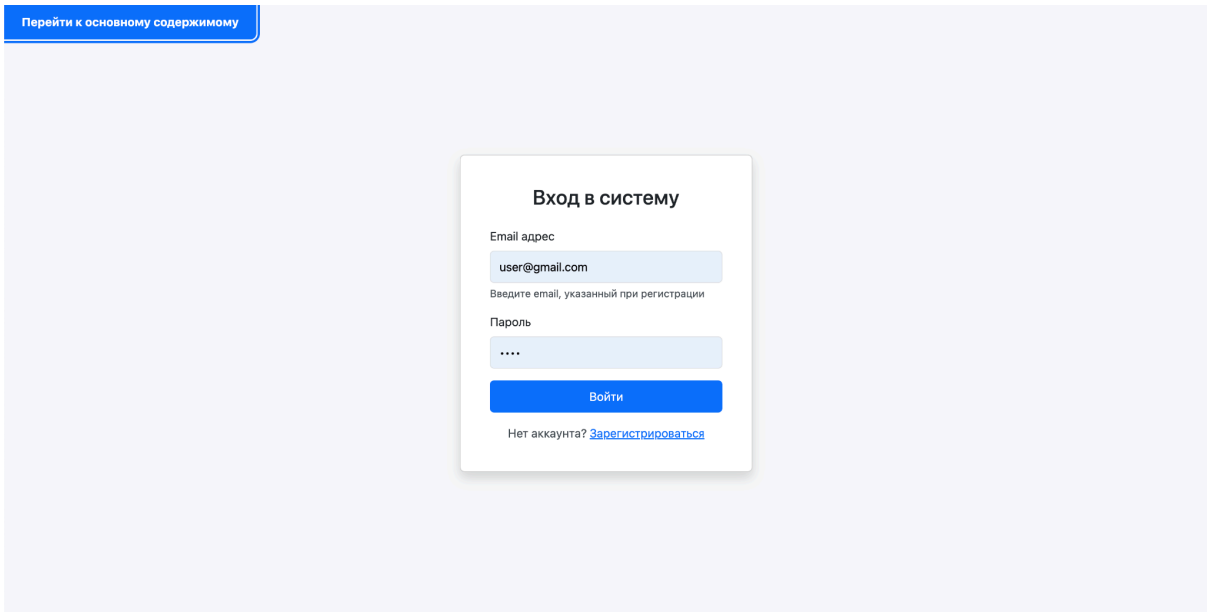
В качестве основы был взят сверстанный в первой и второй лабораторной работе сайт «Мои Финансы».

1. Клавиатурная навигация и Skip-link

На каждую страницу проекта добавлена скрытая ссылка «Перейти к основному содержимому» (класс .skip-link). При обычном просмотре мышью она невидима, однако при первом нажатии клавиши Tab появляется в верхнем левом углу экрана. По клику (или нажатию Enter) фокус мгновенно переносится на элемент `<main id="main-content">`, минуя всю шапку и навигацию. Это критически важно для пользователей, управляющих сайтом исключительно с клавиатуры.

Дополнительно в CSS переопределен стиль фокуса: псевдокласс `:focus-visible` рисует чёткую синюю обводку (`outline: 3px solid #0d6efd`) на всех интерактивных элементах, при этом при клике мышью обводка не появляется (`:focus:not(:focus-visible)`). Для прокручиваемой таблицы транзакций добавлен атрибут `tabindex="0"`, чтобы клавиатурный пользователь мог перейти к ней и прокрутить содержимое стрелками.

Рис. 1. Skip-ссылка, появляющаяся при нажатии Tab на странице входа



2. Семантическая разметка

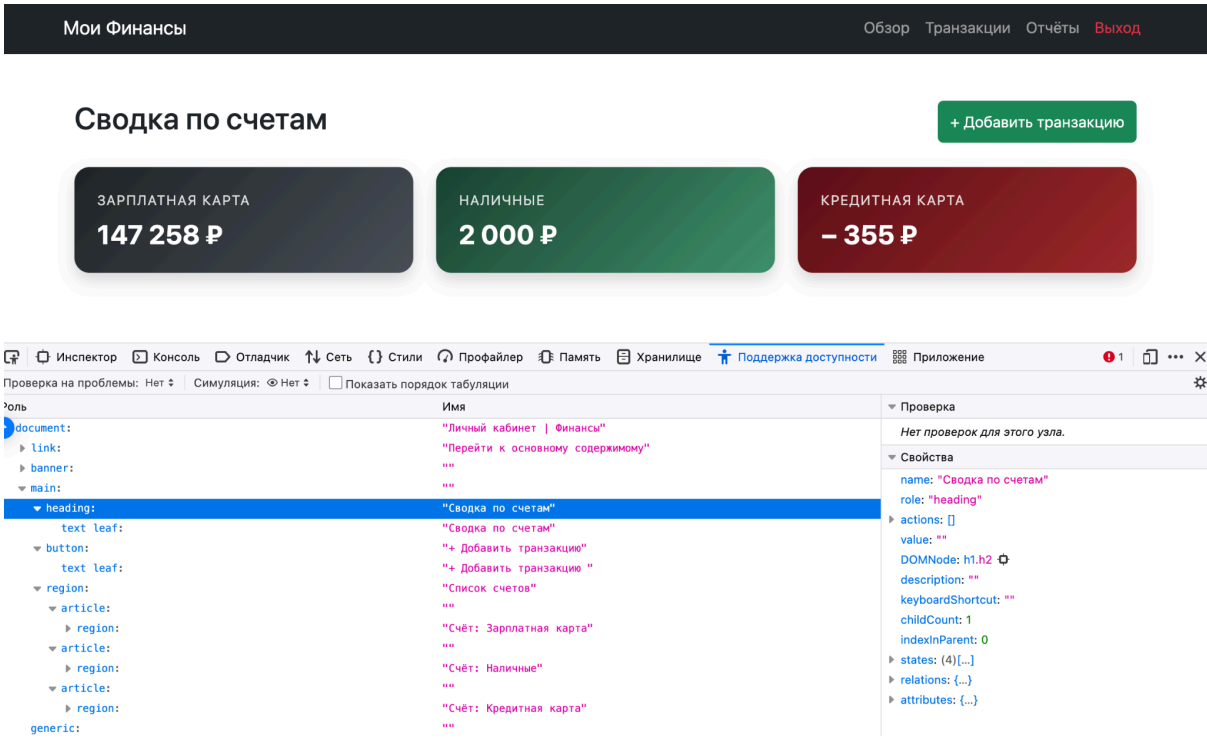
Вся разметка проверена и приведена к семантическому стандарту HTML5:

Элемент	Назначение	Где применён
<header>	Шапка сайта с навигацией	index, transactions, reports
<nav aria-label="...">	Навигационные блоки с понятной подписью	Все страницы с меню
<main id="main-content">	Основное содержимое страницы	Все 6 страниц
<section aria-label="...">	Логические секции (фильтры, счета)	index, transactions
<article aria-labelledby="...">	Самостоятельные блоки (карточки банков)	integrations
<fieldset> + <legend>	Группировка радио-кнопок «Тип операции»	Модальное окно (index)

<caption> (visually-hidden)	Описание таблицы для screen-reader	transactions
scope="col" в <th>	Связь заголовков таблицы с данными	transactions

Ранее некоторые заголовки использовали тег <h2> внутри <article>, а радио-кнопки не были сгруппированы. Теперь структура документа корректна и последовательна, что подтверждается проверкой через инструмент Accessibility Inspector.

Рис. 2. Дерево доступности (Accessibility Tree) страницы index.html в Firefox DevTools



3. ARIA-атрибуты

Для элементов, которым семантики HTML недостаточно, добавлены ARIA-атрибуты:

Атрибут	Где используется	Зачем
aria-label	Кнопки «Применить фильтр», «Подключить СберБанк», поля поиска	Понятное описание действия для screen-reader, когда визуальный текст недостаточен

aria-labelledby	Секции отчётов, карточки интеграций, модальное окно	Связывает элемент с его заголовком
aria-describedby	Поля email и пароля	Связывает поле с подсказкой (form-text)
aria-required="true"	Все обязательные поля форм	Сообщает screen-reader, что поле обязательно
aria-live="assertive"	Блоки ошибок (#loginError, #registerError)	Немедленно озвучивается screen-reader при появлении ошибки
aria-live="polite"	Toast-уведомления, <tbody>, контейнер счетов	Озвучивается при следующей паузе, не прерывая пользователя
aria-atomic="true"	Live-области	Перечитывать всё содержимое блока, а не только изменённую часть
aria-current="page"	Активный пункт навигации	Сообщает, на какой странице пользователь находится
aria-controls / aria-expanded	Кнопка-гамбургер навигации	Стандарт Bootstrap, проверено наличие
role="progressbar" + aria-valuenow/min/max	Прогресс-бары бюджетов	Озвучивание процента использования бюджета
aria-disabled="true"	Кнопка «Отключено» (Tinkoff)	Дублирует HTML-атрибут disabled для ассистивных технологий

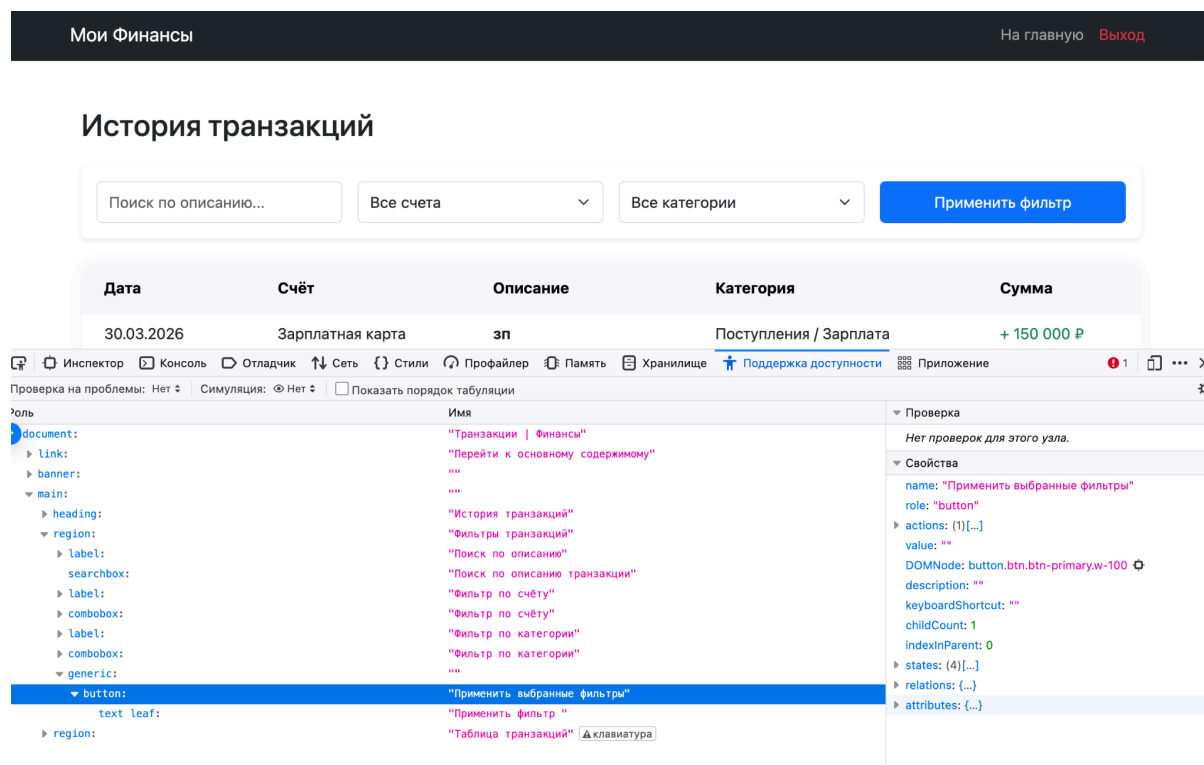


Рис. 3. Свойства доступности кнопки «Применить фильтр» в Firefox Accessibility Inspector

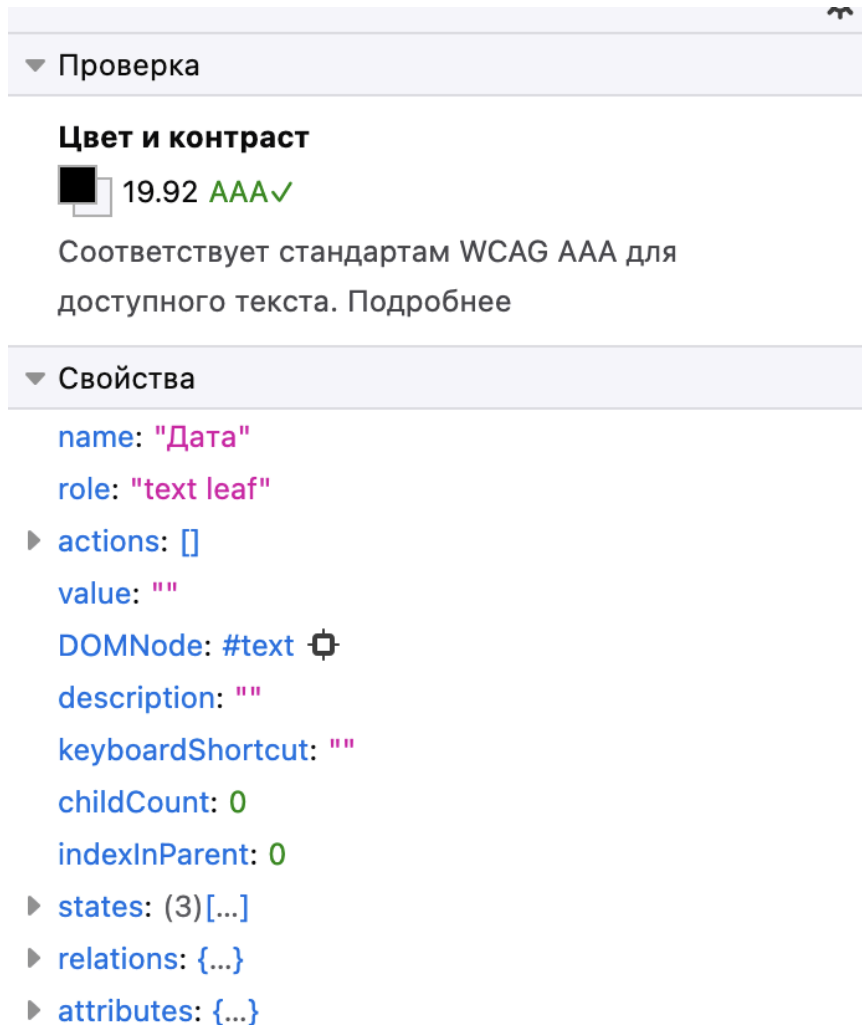
4. Контрастность и стилизация

Согласно принципу воспринимаемости, были внесены следующие визуальные улучшения:

- Переопределен `.text-muted`: стандартный Bootstrap-цвет `#6c757d` заменён на `#495057`, что обеспечивает коэффициент контраста $\geq 4.5:1$ на белом фоне (требование WCAG AA).
- Прогресс-бары получили `font-weight: 700` и увеличенный `font-size`, чтобы процент был читаемым на цветном фоне.
- Минимальный размер интерактивных элементов: для `.btn`, `.form-control` и `.form-select` установлен `min-height: 44px` — рекомендация WCAG для удобного нажатия на мобильных устройствах.
- Кнопка vs ссылка: ссылкам навигации добавлено подчеркивание при наведении и фокусе (`text-decoration: underline`), чтобы визуально отличать их от кнопок.

- С стиль `aria-invalid`: для полей с ошибкой валидации в CSS описан стиль с красной рамкой (`border-color: #dc3545`), что даёт визуальную обратную связь.

Рис. 4. Результат проверки контрастности в Firefox DevTools



5. Обработка ошибок через `aria-live` вместо `alert()`

В предыдущей версии все ошибки авторизации и регистрации выводились через встроенную функцию `alert()`, которая блокирует интерфейс и плохо воспринимается ассистивными технологиями.

Теперь на страницах `login.html` и `register.html` добавлены скрытые блоки:

```
<div id="loginError" class="alert alert-danger d-none" role="alert"
aria-live="assertive"></div>
```

В JavaScript написана функция `showError(containerId, message)`, которая делает блок видимым, вставляет текст ошибки и вызывает `el.focus()` — так screen-reader мгновенно озвучит проблему, а зрячий пользователь увидит красное уведомление прямо в форме.

Аналогично при фильтрации транзакций добавлена функция `announceToSR(message)`, которая создает невидимый `<div aria-live="polite">` и вставляет туда текст «Найдено транзакций: N» — screen-reader озвучит результат, не прерывая пользователя.

Рис. 5. Вывод ошибки авторизации в aria-live блоке вместо `alert()`

The image shows a login form titled "Вход в систему". At the top, there is a red error message box that says "Неверный Email или пароль". Below this, there is a label "Email адрес" followed by a text input field containing "user@gmail.com". Underneath the email field is a hint text: "Введите email, указанный при регистрации". Below the email field is a label "Пароль" followed by a password input field with four dots. At the bottom of the form is a blue button labeled "Войти". Below the button, there is a link: "Нет аккаунта? [Зарегистрироваться](#)".

6. Тег `<noscript>` и кроссбраузерность

На все страницы добавлен тег:

`<noscript>`

```
<div class="alert alert-danger text-center m-3" role="alert">
```

Для работы приложения необходимо включить JavaScript.

</div>

</noscript>

Это обеспечивает принцип надёжности: даже если JavaScript отключен, пользователь получит понятное сообщение вместо пустого экрана. Также на все страницы добавлен мета-тег <meta name="description"> для SEO и вспомогательных технологий, а также проверено наличие <meta name="viewport"> для корректной работы на мобильных устройствах.

7. Проверка с помощью Google Lighthouse

Рис. 6. Результат аудита Lighthouse для страницы index.html

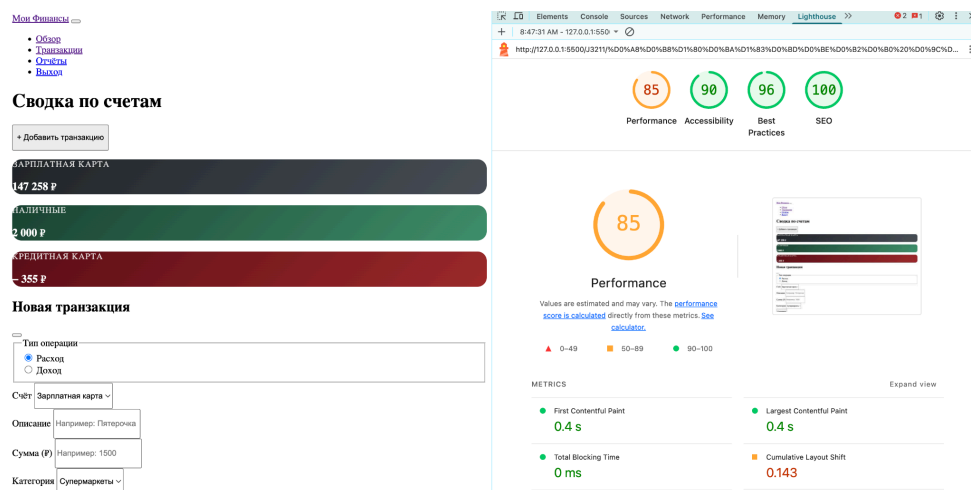
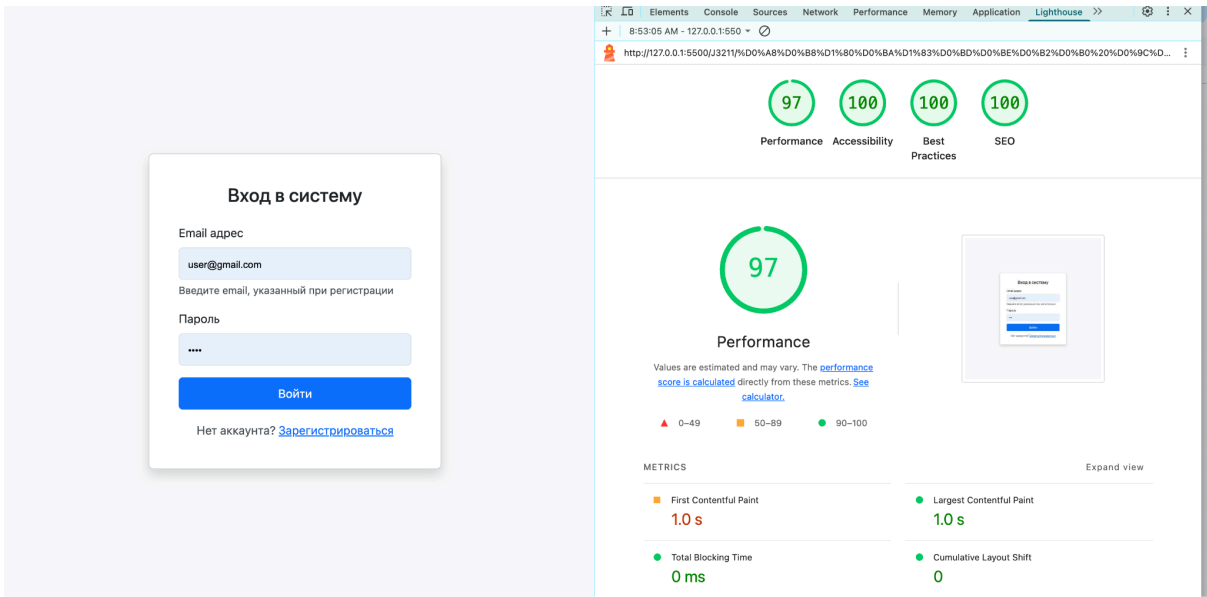


Рис. 7. Детализация пройденных проверок Lighthouse

PASSED AUDITS (20)		Hide
●	[aria-→] attributes match their roles	▼
●	[aria-hidden="true"] is not present on the document <body>	▼
●	[role]s have all required [aria-→] attributes	▼
●	[role] values are valid	▼
●	[aria-→] attributes have valid values	▼
●	[aria-→] attributes are valid and not misspelled	▼
●	Buttons have an accessible name	▼
●	[user-scalable="no"] is not used in the <meta name="viewport"> element and the [maximum-scale] attribute is not less than 5.	▼
●	ARIA attributes are used as specified for the element's role	▼
●	Elements use only permitted ARIA attributes	▼
●	Background and foreground colors have a sufficient contrast ratio	▼
●	Document has a <title> element	▼
●	<html> element has a [lang] attribute	▼
●	<html> element has a valid value for its [lang] attribute	▼

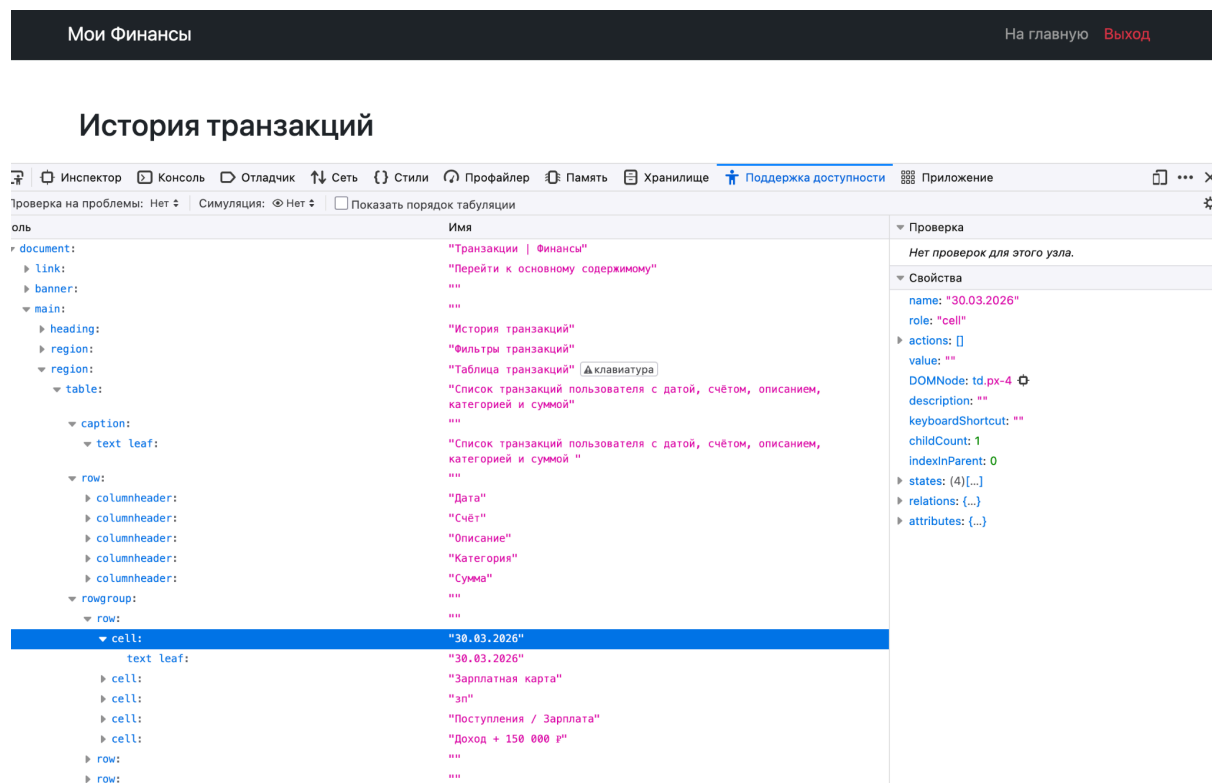
Рис. 8. Результат аудита Lighthouse для страницы login.html



PASSED AUDITS (21)		Hide
●	[aria-*] attributes match their roles	▼
●	[aria-hidden="true"] is not present on the document <body>	▼
●	[role]s have all required [aria-*] attributes	▼
●	[role] values are valid	▼
●	[aria-*] attributes have valid values	▼
●	[aria-*] attributes are valid and not misspelled	▼
●	Buttons have an accessible name	▼
●	Form elements have associated labels	▼
●	[user-scalable="no"] is not used in the <meta name="viewport"> element and the [maximum-scale] attribute is not less than 5.	▼
●	ARIA attributes are used as specified for the element's role	▼
●	Elements use only permitted ARIA attributes	▼
●	Background and foreground colors have a sufficient contrast ratio	▼
●	Document has a <title> element	▼
●	<html> element has a [lang] attribute	▼

8. Проверка с помощью Firefox Accessibility Inspector

Рис. 9. Дерево доступности (Accessibility Tree) в Firefox для transactions.html



Вывод

В результате выполнения домашней работы ранее созданный интерфейс веб-приложения «Мои Финансы» был успешно доработан в соответствии с принципами веб-доступности (WCAG 2.1). Корректность внесённых изменений подтверждена инструментами Firefox Accessibility Inspector и Google Lighthouse.

В ходе работы были изучены и применены на практике следующие подходы и технологии:

- Клавиатурная навигация: Добавлена skip-ссылка для быстрого перехода к основному содержанию. Настроены стили фокуса через псевдокласс `:focus-visible`, что позволяет пользователю, работающему без мыши, всегда видеть, на каком элементе находится курсор. Получено понимание того, как важно обеспечить полноценное управление интерфейсом с клавиатуры.

- Семантическая разметка: Структура всех страниц приведена к стандарту HTML5 с использованием тегов `<main>`, `<nav>`, `<section>`, `<article>`, `<fieldset>`, `<legend>` и `<caption>`. Изучено, как правильная семантика формирует дерево доступности (Accessibility Tree), которое используется экранными читалками для навигации по странице.
- ARIA-атрибуты: Освоена спецификация WAI-ARIA. На практике применены атрибуты `aria-label`, `aria-labelledby`, `aria-describedby`, `aria-required`, `aria-live`, `aria-atomic`, `aria-current`, `aria-hidden` и атрибут `role`. Получено понимание того, что ARIA дополняет семантику HTML там, где стандартных тегов недостаточно, и помогает ассистивным технологиям корректно интерпретировать динамический контент.
- Контрастность и визуальное оформление: Проверены и скорректированы цветовые соотношения текста и фона для достижения коэффициента контраста не менее 4.5:1 (требование WCAG AA). Установлен минимальный размер интерактивных элементов 44×44 пикселя для удобства использования на сенсорных устройствах. Изучено различие между визуальным оформлением кнопок и ссылок.
- Обработка ошибок для screen-reader: Стандартные вызовы `alert()` заменены на встроенные в страницу блоки с атрибутом `aria-live="assertive"`, которые мгновенно озвучиваются экранными читалками при появлении ошибки. Реализована вспомогательная функция `announceToSR()` для озвучивания результатов фильтрации. Получено понимание разницы между значениями `assertive` (немедленное оповещение) и `polite` (оповещение при паузе).
- Инструменты проверки доступности: Освоена работа с двумя инструментами аудита. Google Lighthouse позволяет провести автоматизированную проверку страницы и получить числовую оценку доступности с детализацией пройденных и не пройденных проверок. Firefox Accessibility Inspector даёт возможность исследовать дерево доступности, проверять контрастность отдельных элементов и выполнять поиск проблем по всей странице. Совместное использование обоих инструментов обеспечивает наиболее полное покрытие проверки.